Taste Match SE — Every query that runs

Novelty Library v2.19 · admin reference · documentation only (not imported by the app)

Three kinds of "query" exist: **database reads**, **in-browser searches**, and **public-web lookups**. Taste Match makes no database writes.

A. Database reads (Supabase, read-only)

#	When	Call	Reads	Notes
A1	Taste Match page, Live Rankings, review pages, author pages open	`supabase.from('public_community_reviews').select('*').eq('is_published', true).order('published_on', {ascending:false, nullsFirst:false}).order('review_no', {ascending:false})` (`lib/reviews.ts › fetchReviews`)	Published reviews	Privacy-safe view: no reviewer email or contact details. Result is reused by every Taste Match feature on that page.
A2	Taste Match page, after A1	`supabase.from('community_review_feedback').select('review_id,rating').in('review_id', <published ids>)`	Reader star ratings	Used only to compute the NL rating. If it fails, matching continues without it.
A3	Live Rankings	`supabase.from('community_review_feedback')` (same table)	Reader star ratings	Same purpose for the rankings list.
A4	Review page "More like this"	`supabase.from('community_review_feedback')` for the three suggested reviews only	Reader star ratings	Shows NL beside each suggestion.

Nothing else touches the database. Matching never writes, never logs a search, never stores a choice.

Security note. The planned lockdown script (`sql/NL_SQL_LOCKDOWN_run_after_adsense.sql`) re-creates the public views as definer views, so A1 keeps working after it is run. A2–A4 read `community_review_feedback`; check that table's public policy before tightening it.

B. In-browser searches (no network)

All of these run over the already-loaded list from A1 (+ ratings from A2).

B1. Taste Match picks — `rankTmai40(books, {description, moods, lovedId, language, verdicts}, 3)`

- Input wish = description + mood words + loved-book profile.
- Filters: language equals, verdict in chosen, not the loved book.
- Score = 62% TF-IDF cosine + 38% weighted metrics (see Architecture §7). Gate at 1.5%. Tie-break NL → R/W → title.

B2. Surprise me

`rankTmai40(..., 12)` → random one. No wish: random from Perfection / Go for it (in language), else any.

B3. Live Rankings free description — `rankTmai40(books, {description}, 5)`

B4. More like this — `rankTmai40(books, {lovedId: <this review>}, 3)`

B5. Exact-first field search — `strictSearch(books, field, query)`

Field ∈ genre, language, traits, author. Tiers: 100 exact, 92 equivalent, 85 phrase, 80 contained, 75 all words, 60 typo.

B6. Wider matches — `broadSearch(books, text, {exclude, minScore})`

TF-IDF + cosine, minimum 5% (8% for variants of a direct search).

B7. Combined — `runSearch(books, field, strictQuery, moreText)`

Modes: none / direct / description / both.

B8. Fall-backs

- `didYouMean` — nearest real values for that field (up to 3).
- `nearestBooks` — wide search ≥ 0.5%, else reader favourites.
- `popularValues` — the quick-pick chips (most common values for a field).

B9. Remembered-book search — `rememberedSearch(books, text, 3)`

Weights: title 3, author 2.5, genre/traits/review words 1.6 × rarity. Needs ≥ half the words to hit or a close whole title. Minimum 22%.

B10. Match profile (display only)

Counts first genres and traits across the top 3 picks. No scoring.

B11. Admin settings lab — `labSearch(books, text, config, minScore, 10)`

Runs the TF-IDF engine with admin-chosen weights next to the live ones. Sandbox: nothing is saved or shown to readers.

B12. Intro numbers

Distinct counts of books, authors, languages and first genres in the published list.

C. Public-web lookups (from the reader's browser, no API keys)

#	Trigger	URL	Purpose
C1	Remembered-book box has no local result; ≥ 4 chars; 600 ms pause	`https://openlibrary.org/search.json?q=<text>&limit=3&fields=title,author_name,first_publish_year`	"Did you mean" for titles that are not in our reviews yet
C2	Author page opens	`https://en.wikipedia.org/w/api.php?action=query&generator=search&gsrsearch= writer author&gsrlimit=4&prop=extracts	description
C3	Author page opens	`https://openlibrary.org/search.json?author=<name>&sort=editions&limit=25&fields=title,first_publish_year,author_name,edition_count`	Other titles and publication years

Acceptance rules: a Wikipedia page is used only if its title contains the author's name and its first 600 characters mention a writer-type role; Open Library works are used only if the author name matches. Each request has a 6-second timeout and fails silently. Results are cached in memory only. The CSP in `vercel.json` must keep `https://openlibrary.org` and `https://en.wikipedia.org` in `connect-src`.

Privacy: C1–C3 reveal the typed text or author name and the reader's IP address to those sites. Nothing goes to Novelty Library's servers.

D. Quick "which query did that?" guide

You see...	Query
Three picks with "Echoes your words"	B1
Results after typing in the Live Rankings description box	B3
"More like this" under a review	B4
Genre/language/traits/author box results	B5 + B6 (B7)
"Did you mean..."	B8
"Can't spell it?" results	B9 (C1 when empty)
"About the author" note	C2 + C3
NL rating beside a book	A2
Counts in "What is Taste Match SE?"	B12